Lia-Script

With Lia, we try to implement an extended Markdown format that should enable everyone to create, share, adapt, translate or correct and extend online courses without the need of being a web-developer.

See the online rendered version at: https://liascript.github.io/course/?https://raw.githubusercontent.com/liaScript/docs/master/README.md

Click on the (ear) button at the navigation bar to switch between spoken and plain text mode format. And use the arrows-buttons at the top for navigating.

Lia-Script is an extended Markdown format for writing interactive online courses. Imagine all schoolbooks, technical or scientific literature could become open-source projects and more interactive … with collaborating teachers and students …

- easy to share adapt and translate
- no additional software required, everything is implemented in JavaScirpt/Elm and runs directly within the browser (online)
- automatic conversion to epub, pdf, …

Everything that is required is simple text-editor and a web-browser. Or you start directly to create and share your course on github. The entire parsing and transformation of Lia-Markdown to any other format is done within the browser at client-side.

Tools

There are currently 2 Plugins for the Atom Editor available, which are intended to ease and simplify the development of online course with LiaScript.

liascript-preview: Is a tiny previewer that, if it was toggled, updates the view on your course each time you save your document.

liascript-snippets: If you start typing “lia” in your document you switch on a fuzzy search, that contains a lot of LiaScript help, examples, and snippets.

Markdown-Syntax

This section is intended to give a brief overview on the basic Markdown formatting elements. The only difference to common Markdown at this point is, that every course has to start with a comment, which defines authors, a language and a narrator voice, see https://responsivevoice.org for all supported voices.

Initial LIA-comment-tag for basic definitions:

```
<!--

author:   Andre Dietrich

email:    dietrich[at]ivs.cs.uni-magdeburg.de

version:  1.0.0

language: en

narrator: US English Female

script:   javascript resourse url

script:   another javascript resourse url

link:     some css stuff
          and some more css

-->
```

Structuring

A course is structured as any other Markdown document with starting hash-tags, whereby the number of hash-tags is used to define the hierarchy.

```
# Main Title

 ...

## Section Title 1

 ...

### Subsection Title

 ...
## Section Title 2
```

Text-Formatting

We tried to use the github flavored Markdown style for simple formatting with some additional elements.

italic → *italic*

bold → **bold**

bold and italic *** → ***bold and italic

also italic → *also italic*

__also bold__ → **also bold**

___also bold and italic___ → ***also bold and italic***

~strike~ → ~~strike~~

These exceptions are for example underline and its combination with strike through or the application of superscript. If you, for example, superscript superscript you can get even smaller.

~~underline~~ → underline

~~~strike and underline~~~ → ~~strike and underline~~

^superscript^ → superscript ^^superscript^^ ^^^superscript^^^

Combinations

As you can see from the examples, you can combine all elements freely.

bold _bold italic_ → **bold *italic***

~bold strike~ ~~bold underline~~ → **~~bold strike~~** **bold underline**

~italic strike~ ~~italic underline~~ → ***~~italic strike~~*** *italic underline*

Escape Characters

*, ~, _, #, {, }, [,], |, `, $

If you want to use multiple stars, hash-tags, or other syntax elements within your script without applying their functionality, then you can escape them with a starting backslash.

Symbols

One thing that we missed in standard Markdown, was an implementation for arrows. The parenthesis shows, how arrows are defined in our Markdown implementation with their result on the right (these symbols are generated automatically for you).

(`->`) →, (`->>`) ↠, (`>->`) ↣, (`<-`) ←, (`<-<`) ↢, (`<<-`) ↞, (`<->`) ↔, (`=>`) ⇒, (`<=`) ⇐, (`<=>`) ⇔

(`-->`) ⟶, (`<--`) ⟵, (`<-->`) ⟷, (`==>`) ⟹, (`<==`) ⟸, (`<==>`) ⟺

(`~>`) ⇝, (`<~`) ⇜
~~~

But you can also use some basic smileys. We will try to extend this partial support in the future.

`:-)` 🙂, `;-)` 😉, `:-D` 😀, `:-O` 😮, `:-(` 🙁, `:-|` 😐, `:-/` 😕, `:-P` 😛, `:-*` :-*, `:')` 😂, `:'(` 😢

References

The next section shows how external resources can be integrated.

Simple Links

There are two ways of adding links to a Markdown document, either by inlining the url directly or you can name it, as shown in listing 2, by applying the typical brackets and parenthesis notation, the optional info is put in double quotes at the end of the url.

1. example of an url-link → http://goo.gl/fGXNvu

   text-formatting can be applied also (`*** http://goo.gl/fGXNvu ***`) → ***http://goo.gl/fGXNvu***

2. naming the link (`[title](http://goo.gl/fGXNvu "optional info")`) → title

Images and Movies

Images are marked with a starting exclamation mark before the link, audio by a starting question mark and movies are made of images and sound, that is why you combine both marks `!?`. Defining ressources this way shows at least the links correctly in other Markdown parsers or on github.

Image-notation: `![alt-text](image-url "some info")`

- url: `![image](https://upload.wikimedia.org/wikipedia/commons/d/d0/Creative-Tail-Animal-lion.svg "a picture of a lion")`
- 

  a picture of a lion
- relative path: `![image](img/lenna.jpg)`
- *image*

  beautiful Lenna

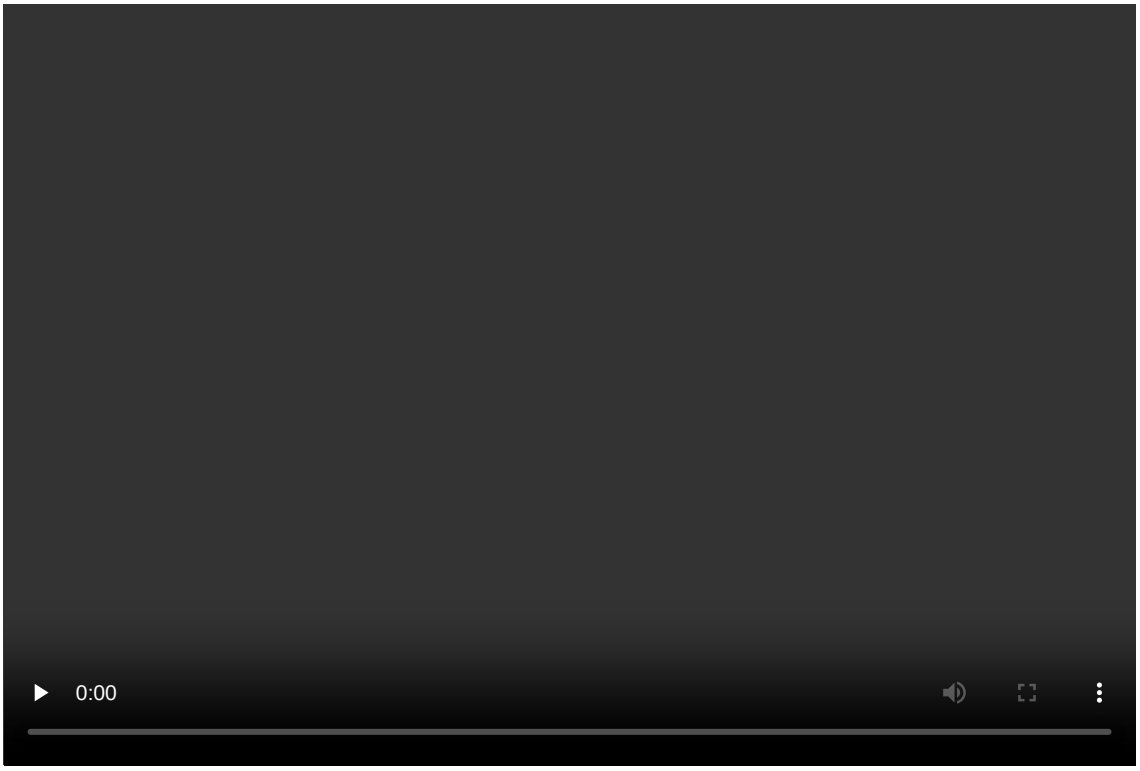
Audio-notation: `?[alt-text](audio-url "some info")`

- `?[a horse](https://www.w3schools.com/html/horse.mp3 "hear a horse")`
- 0:00 / 0:00

  ♪ horse.mp3

  hear a horse

Movie-notation: `!?[alt-text](movie-url)`

- YouTube: `!?[movie](https://www.youtube.com/embed/XsNk5aOpqUc?&autoplay=1)`
- 

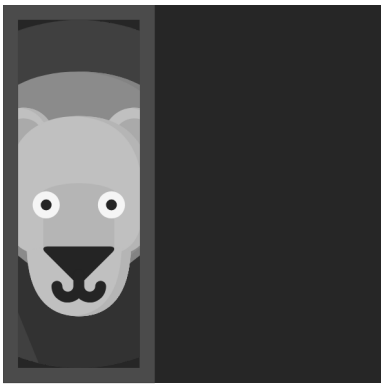
- See also http://www.google.com/support/youtube/bin/answer.py?hl=en&answer=56107 to get an overview on how a YouTube link has to be formatted to add a starting and/or end point, autoplay, subtitles, and other options.
- relative path: `!?[movie](vid/math.mp4)`
- 

Styling

Adding CSS elements or classes or any other HTML setting to an image or to any other Markdown element is implemented via a trailing comment-tag, everything within this comment is treated as a HTML attribute, so that it can also be used to apply graphical filters of for positioning.

```
![image](...Creative-Tail-Animal-lion.svg)<!--
style = "width: 100px;
        border: 10px solid;
        filter: grayscale(100%);"

class = "animated infinite bounce"
-->
```

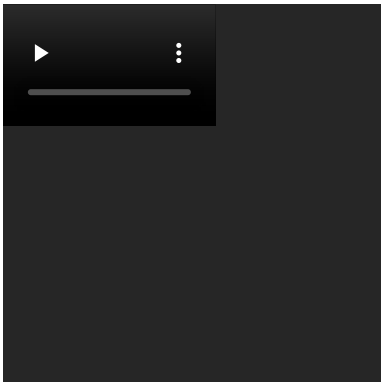






The same technique can also be applied to style and format movies and other inline elements, such as links, words, symbols, or code...








`:-) <!-- class = "animated infinite bounce" -->` ⟹ 🙂

`[google](www.google.de) <!-- style = "color: red;" -->` ⟹ google

Block-Styling

Settings for entire blocks can be set with a **starting** comment that includes all required HTML-attributes and can even contain animation settings. These can be used to highlight specific elements of your slides.

```
<!-- class = "animated rollIn" style = "animation-delay: 3s; color: purple" -->
The whole text-block should appear in purple color and with a wobbling effect.
Which is a **bad** example, please use it with causion ...
~~ ;-) ~~ <!-- class = "animated infinite bounce" style = "color: red;" -->
```

The whole text-block should appear in purple color and with a wobbling effect. Which is a **bad** example, please use it with causion … 😉

Lists & Tables

Within the following part enumerations, itemizations and tables are presented, actually with no difference to basic Markdown, so you can skip this section, if you are already familiar with it.

Unordered Lists

To define unordered list, starting stars, pluses, and minuses can be used and mixed. If one point has more than one line, you can also use newlines, but with spaces at the beginning. Paragraphs can be included in the same way, by using two newlines.

Markdown-format:

```
* alpha
+ *beta*
- gamma
  and delta

  new Paragraph
```

Result:

- alpha
- *beta*
- gamma and delta

  new Paragraph

Ordered Lists

Ordered list start with a number and a dot. As you can see from the example, the number does not count at the moment, the generated list will always count by the order of appearance. And it is also possible to mix lists with other lists and elements freely.

Markdown-format:

```
0. alpha
2. **beta**
1. * gamma
   * delta
   * and epsilon
3. probably zeta
```

Result:

0. alpha
2. **beta**
1. - gamma
   - delta
   - and epsilon
3. probably zeta

Tables

Tables, we hope, are self-explanatory. The second line is used to define a table header as well as the alignment of the column, which is indicated by the colon. The default is left alignment.

Markdown-format:

```
| Tables            | Are           | Cool  |
| ----------------- |:-------------:| -----:|
| *** col 3 is ***  | right-aligned | $1600 |
| ** col 2 is **    | centered      |   $12 |
| * zebra stripes * | are neat      |    $1 |
```

Result:

Tables	Are	Cool
col 3 is	right-aligned	$1600
col 2 is	centered	$12
zebra stripes	are neat	$1

Blockquotes

Markdown-format:

```
<!--
style="font-size: 18px; font-style: italic; width: 500px; margin: 0.25em 0;"
-->
> Blockquotes are very handy in email to emulate reply text.
> This line is part of the same quote.

Quote break.

> This is a very long line that will still be quoted properly when it wraps...
```

Result:

> *Blockquotes are very handy in email to emulate reply text.*
> *This line is part of the same quote.*

Quote break.

> This is a very long line that will still be quoted properly when it wraps. Oh boy let's keep writing to make sure this is long enough to actually wrap for everyone. Oh, you can *put* **Markdown** into a blockquote.

Next to simple paragraphs also any other block element can be included …

```
> <!-- style="font-size: 18px; font-style: italic; width: 500px; margin: 5.25em
0;" -->
> | Tables            | Are           | Cool  |
> | ----------------- |:-------------:| -----:|
> | *** col 3 is ***  | right-aligned | $1600 |
> | ** col 2 is **    | centered      |   $12 |
> | * zebra stripes * | are neat      |    $1 |
```

Result:

> | *Tables* | *Are* | *Cool* |
> |---|---|---|
> | *col 3 is* | *right-aligned* | *$1600* |
> | *col 2 is* | *centered* | *$12* |
> | *zebra stripes* | *are neat* | *$1* |

HTML

You can also use raw HTML in your Markdown, and it'll mostly work pretty well.

```
Test **bold** and <b> HTML bold</b> works also inline

<dl>
  <dt style="color: red">Definition list</dt>
  <dd>Is something people use sometimes.</dd>

  <dt><b>Markdown in HTML</b></dt>
  <dd>Does *not* work **very** well. Use HTML <em>tags</em>.</dd>
</dl>
```

Test **bold** and **HTML bold** works also inline

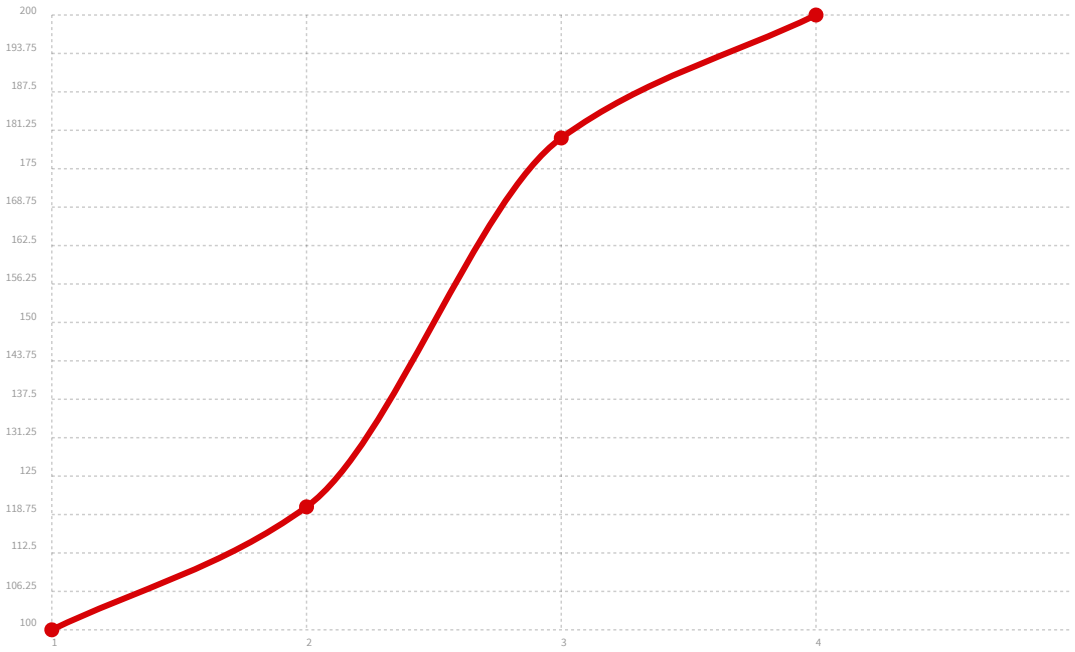
Definition list
Is something people use sometimes.

Markdown in HTML
Does *not* work **very** well. Use HTML *tags*.

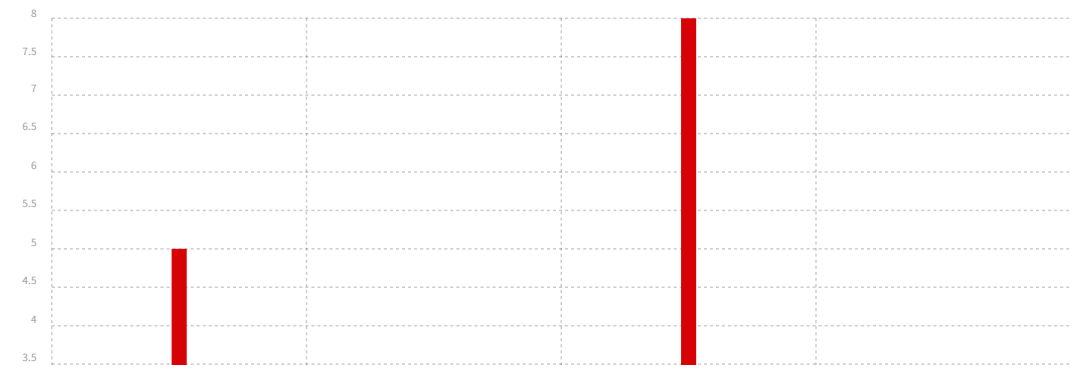
HTML & JavaScirpt

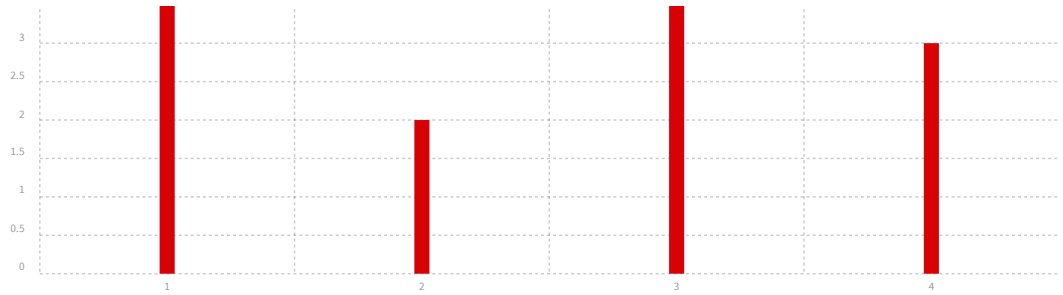
In contrast to common Markdown-Parsers it is also possible to include and execute javascript code. If you combine it with your HTML elements, you are free to integrate whatever you want.

```
<div class="ct-chart ct-golden-section" id="chart"></div>
<script>
    // Initialize a Line chart in the container with the ID chart
    new Chartist.Line('#chart', {
        labels: [1, 2, 3, 4],
        series: [[100, 120, 180, 200]]
    });
</script>
```



TypeError: Converting circular structure to JSON --> starting at object with constructor 'f' | property 'container' -> object with constructor 'HTMLDivElement' --- property '__chartist__' closes the circle





TypeError: Converting circular structure to JSON --> starting at object with constructor 'f' | property 'container' -> object with constructor 'HTMLDivElement' --- property '__chartist__' closes the circle

Note, you have to include all required JavaScirpt-resourses in the initial comment after the script definition. And by combining this feature with LiaScript effects, you can build even more sophisticated courses.

Footnotes

There are two types of footnotes, either inline or divided ones (into two parts). The the footnotbody is only showed in textbook mode, or if you click on the marker.

1. Inline Footnote [1] ⇒ `[^1](explanation in one line)`
2. Divided into a marker [2] ⇒ `[^2]`, that can appeare every where and an explanation at the bottom of a section.

```
[^2]: This is an explanation, than
      can consist of multiple blocks.
```

---

[1] explanation in one line

[2] This is an explanation, than can consist of multiple blocks.

Math-Mode

via KaTex http://katex.org

Inline math-mode `$ \frac{a}{\sum{b+i}} $` → $\frac{a}{\sum{b+i}}$

Multi-line math-mode can be applied by double dollars `$$ formula $$`

$$\frac{a}{\sum b+i}$$

We apply KaTeX for math-formatting, see the documentation at www.katex.org.

A formula can be either inline with single dollars.

Or multi-line by using the double dollar notation.

Syntax Highlighting

The following section shows a three types of code blocks, simple inline and block-code as well as interactive block-code, as an extension to common Markdown.

Inline-Code

Inline code via ` enter some code in here 1###$& ` → `enter some code in here 1###$&`

Block-Code

Syntax highlighting is enabled with highlight.js. Blocks of code are either by lines with three back-ticks ``` and an identifier for the language. See a complete list of examples and how to write language names at the highlight.js demo page.

```` ``` `language (python, c, javascript, ...)` ````

code …

```
```

C example:

```c
#include "test.h"

int main () {
    printf("this is an example\n");
    return 0;
}
```

Python example:

```python
import math

def lia_sqrt(val):
    return math.sqrt(val) + 22
```

Interactive Code

Why should code examples not be interactive and editable, especially if it is JavaScript or any other language that has been ported to it? Simply add the required resources to the initial comment with keyword `script`.

1. Add resource to main-comment: `script: url.js`
2. Add a trailing script-tag to your code: `<script>@input</script>`
3. A project with multiple files can be realized with `@input(0)`, `@input(1)`, …, `@input(n)`.

And add an additional script tag to the end of your language definition with an `@input` macro. This element is afterwards substituted with your code and executed. We provide some basic examples within the following section.

Use the `@input` macro as a parameterized function in projects. The number defines the the file, starting from 0.

JavaScript

Click on the run-button to execute the script or double-click on the code to edit it and to change the output …

Double-click on the code to switch to edit mode and double-click to get out:

```javascript
var i=0;
var j=0;
var result = 0;

```
```

```
for(i = 0; i<1000; i++) {
    for(j = 0; j<i; j++) {
        result += j;
    }
}
// the last statement defines the return statement
result;
```

```
166167000
```

Projects

Mutliple different code snippets can be combined to form a larger project. It requires to wo write them in a row. You can give them names, if you add a second parameter after the highlighting definition. Add a `+` or `-` to the front of your filename, in order to indicate, if it should be visible by default or not.

EvalScript.js

```
let who = data.first_name + " " + data.last_name;

if(data.online) {
  who + " is online"; }
else {
  who + " is NOT online"; }
```

Data.json

```
{
  "first_name" :  "Sammy",
  "last_name"  :  "Shark",
  "online"     :  true
}
```

```
Sammy Shark is online
```

JavaScript Chartist

A drawing example, for demonstrating that any javascript library can be used, also for drawing.

```
// Initialize a Line chart in the container with the ID chart1
new Chartist.Line('#chart1', {
  labels: [1, 2, 3, 4],
  series: [[100, 120, 180, 200]]
});

// Initialize a Line chart in the container with the ID chart2
new Chartist.Bar('#chart2', {
  labels: [1, 2, 3, 4],
  series: [[5, 2, 8, 3]]
});
```

```
[object Object]
```

Computer-Algebra

An example of a Computer-Algebra-System (Algebrit), see http://algebrite.org for more examples:

```
1  x + x
```

```
2*x
```

```
f=sin(t)^4-2*cos(t/2)^3*sin(t)

f=circexp(f)

defint(f,t,0,2*pi)
```

```
-16/5+3/4*pi
```

Elm

```
-- Read more about this program in the official Elm guide:
-- https://guide.elm-lang.org/architecture/user_input/buttons.html

import Html exposing (beginnerProgram, div, button, text)
import Html.Events exposing (onClick)


main =
  beginnerProgram { model = 0, view = view, update = update }


view model =
  div []
    [ button [ onClick Decrement ] [ text "-" ]
    , div [] [ text (toString model) ]
    , button [ onClick Increment ] [ text "+" ]
    ]


type Msg = Increment | Decrement


update msg model =
  case msg of
    Increment ->
      model + 1

    Decrement ->
      model - 1
```

C++

Teaching other language-basics is also possible, for this example we applied JSCPP to run simple C++ programs:

```
#include <iostream>
using namespace std;

int main() {
    int a = 120;
    int rslt = 0;
    for(int i=1; i<a; ++i) {
        rslt += i;
        cout << "rslt: " << rslt << endl;
    }
    cout << "final result = " << rslt << endl;
    return 0;
}
```

```
rslt: 1
rslt: 3
rslt: 6
rslt: 10
rslt: 15
rslt: 21
rslt: 28
rslt: 36
rslt: 45
rslt: 55
rslt: 66
rslt: 78
rslt: 91
rslt: 105
rslt: 120
rslt: 136
rslt: 153
rslt: 171
rslt: 190
rslt: 210
rslt: 231
rslt: 253
rslt: 276
rslt: 300
rslt: 325
rslt: 351
rslt: 378
rslt: 406
rslt: 435
rslt: 465
rslt: 496
rslt: 528
rslt: 561
rslt: 595
rslt: 630
rslt: 666
rslt: 703
rslt: 741
rslt: 780
rslt: 820
rslt: 861
rslt: 903
rslt: 946
rslt: 990
rslt: 1035
rslt: 1081
rslt: 1128
rslt: 1176
rslt: 1225
rslt: 1275
rslt: 1326
rslt: 1378
rslt: 1431
rslt: 1485
rslt: 1540
rslt: 1596
rslt: 1653
```

```
rslt: 1711
rslt: 1770
rslt: 1830
rslt: 1891
rslt: 1953
rslt: 2016
rslt: 2080
rslt: 2145
rslt: 2211
rslt: 2278
rslt: 2346
rslt: 2415
rslt: 2485
rslt: 2556
rslt: 2628
rslt: 2701
rslt: 2775
rslt: 2850
rslt: 2926
rslt: 3003
rslt: 3081
rslt: 3160
rslt: 3240
rslt: 3321
rslt: 3403
rslt: 3486
rslt: 3570
rslt: 3655
rslt: 3741
rslt: 3828
rslt: 3916
rslt: 4005
rslt: 4095
rslt: 4186
rslt: 4278
rslt: 4371
rslt: 4465
rslt: 4560
rslt: 4656
rslt: 4753
rslt: 4851
rslt: 4950
rslt: 5050
rslt: 5151
rslt: 5253
rslt: 5356
rslt: 5460
rslt: 5565
rslt: 5671
rslt: 5778
rslt: 5886
rslt: 5995
rslt: 6105
rslt: 6216
rslt: 6328
rslt: 6441
rslt: 6555
```

```
rslt: 6670
rslt: 6786
rslt: 6903
rslt: 7021
rslt: 7140
final result = 7140
```

Prolog

See the implementation details at: https://curiosity-driven.org/prolog-interpreter

Load Database and Rules:

```
exists(A, list(A, _, _, _, _)).
exists(A, list(_, A, _, _, _)).
exists(A, list(_, _, A, _, _)).
exists(A, list(_, _, _, A, _)).
exists(A, list(_, _, _, _, A)).

rightOf(R, L, list(L, R, _, _, _)).
rightOf(R, L, list(_, L, R, _, _)).
rightOf(R, L, list(_, _, L, R, _)).
rightOf(R, L, list(_, _, _, L, R)).

middle(A, list(_, _, A, _, _)).

first(A, list(A, _, _, _, _)).

nextTo(A, B, list(B, A, _, _, _)).
nextTo(A, B, list(_, B, A, _, _)).
nextTo(A, B, list(_, _, B, A, _)).
nextTo(A, B, list(_, _, _, B, A)).
nextTo(A, B, list(A, B, _, _, _)).
nextTo(A, B, list(_, A, B, _, _)).
nextTo(A, B, list(_, _, A, B, _)).
nextTo(A, B, list(_, _, _, A, B)).

puzzle(Houses) :-
    exists(house(red, english, _, _, _), Houses),
    exists(house(_, spaniard, _, _, dog), Houses),
    exists(house(green, _, coffee, _, _), Houses),
    exists(house(_, ukrainian, tea, _, _), Houses),
    rightOf(house(green, _, _, _, _), house(ivory, _, _, _, _), Houses),
    exists(house(_, _, _, oldgold, snails), Houses),
    exists(house(yellow, _, _, kools, _), Houses),
    middle(house(_, _, milk, _, _), Houses),
    first(house(_, norwegian, _, _, _), Houses),
    nextTo(house(_, _, _, chesterfield, _), house(_, _, _, _, fox), Houses
      ),
    nextTo(house(_, _, _, kools, _),house(_, _, _, _, horse), Houses),
    exists(house(_, _, orangejuice, luckystike, _), Houses),
    exists(house(_, japanese, _, parliament, _), Houses),
    nextTo(house(_, norwegian, _, _, _), house(blue, _, _, _, _), Houses),
    exists(house(_, _, water, _, _), Houses),
    exists(house(_, _, _, _, zebra), Houses).

solution(WaterDrinker, ZebraOwner) :-
    puzzle(Houses),
    exists(house(_, WaterDrinker, water, _, _), Houses),
    exists(house(_, ZebraOwner, _, _, zebra), Houses).
```

```
database loaded
```

Query: (it may take some time 😉)

```
1 solution(WaterDrinker, ZebraOwner)
```

```
Yes: solution(norwegian, japanese)
```

More Examples

We provide a list of templates with more examples that can be used to start developing your own courses. See:

https://liascript.github.io/course/?https://raw.githubusercontent.com/liaScript/templates/master/README.md

Quizzes

Quizzes are an essential element of every online course for students to reflect and test their knowledge. Lia currently supports three different types of quizzes which can be tweaked, if required.

Text Inputs

A text input field is defined simply by a newline and two brackets around the solution word, value or sentence. In the depicted example, the word solution is the solution. If you enter something else, the check will fail.

Markdown-format: `[[solution]]`

Please enter the word *"solution"* into the text-field!

Single-Choice

A single choice quiz can be defined with parenthesis within brackets and an X, which marks the only correct answer option. The additional text is Lia-Markdown again.

Markdown-format:

```
[( )] Wrong
[(X)] This is the **correct** answer
[( )] This is ~~wrong~~ too!
```

Only one element can be selected!

- ◯ Wrong
- ◯ This is the **correct** answer
- ◯ This is ~~wrong~~ too!

Multiple-Choice

A multiple choice quiz can be defined with brackets within brackets and an X, which are used to mark the correct answer option. In contrast to single choice quizzes, there can be multiple selected choices or no one, which is also allowed.

```
[[ ]] Do not touch!
[[X]] Select this one ...
[[X]] ... and this one too!
[[ ]] also not correct ...
```

Multiple of them can be selected, or all, or none of them …

- [ ] Do not touch!
- [ ] Select this one …
- [ ] … and this one too!
- [ ] also not correct …

Hints

To any type of quiz you can add as many hints as you want, which are revealed in order by clicking onto the question mark.

Markdown-format:

```
  [[super]]
  [[?]] another word for awesome
  [[?]] not great or mega
  [[?]] hopefully not that bad
  [[?]] there are no hints left
```

A text input with additional hints:

Solution

And finally, some quizzes might require some more explanations, if they are solved or not. That is why, with additional three opening and three closing brackets you mark the beginning and the end of your solution, which can contain multiple paragraphs, formulas, program code, videos, etc as well as effects (see therefor the next section).

Markdown-format:

```
  [[super]]
  [[?]] hint 1
  [[?]] hint 2
  **********************************************************************

                                {{1}}
You are right, super was the correct answer again

* {2}{super} as an effect
* $\sum x + 3$
* terra

![image](https://upload.wikimedia.org/wikipedia/commons/d/d0/Creative-Tail-Animal
-lion.svg)

  **********************************************************************
```

A quiz with hints and a revealed result.

Effects

There are currently three types of effects, that are supported by liaScript:

1. Inline effects
2. Block effects
3. and a narrator

Every effect is defined by two braces around one or two numbers `{{1-2}}`, which marks the order of their appearance. The second and optional number defines, at which step the element becomes invisible again.

Inline Effects

Inline effects can be used in nearly all liaScript elements, as already mentioned, the first number within curly braces defines the number of appearance while the second part defines those Markdown elements that should be revealed step wise. If you use 0, then the effect will be revealed immediately.

Use this to highlight important facts and to structure your slides (multiple effects can be combined, due to the usage of equal numbers):

```
* no effect here
* but in this line {1}{show *** first ***}
* as well as this one {1-2}{show *** first ***}, which contains two effects
  {2}{![image](https://upload.wikimedia.org/wikipedia/commons/d/d0/Creative-Tail-
  Animal-lion.svg)}
```

- no effect here
- but in this line show ***first***
- as well as this one show ***first***, which contains two effects 

Animations

To define animations and transitions, you can use the animate.css notation, simply place an animation definition within the comment tag after the effects, as it is done within the examples.

See https://daneden.github.io/animate.css for more animation effects.

```
* {0}{ *bounce* } <!-- class = "animated infinite bounce" style =
"animation-delay: 5s;" -->
* {1-2}{zoomIn} <!-- class = "animated zoomIn" -->
* {2-3}{zoomOut} <!-- class = "animated zoomOut" -->
* {3}{![image](https://upload.wikimedia.org/wikipedia/commons/d/d0/Creative-Tail-
Animal-lion.svg)} <!-- class = "animated rubberBand" -->
```

- `bounce`
- `zoomIn`
- `zoomOut`
- 

Block Effects

Block effects as animations are defined similarly to inline effects, just with one additional newline after the effect definition. The following Markdown block is then entirely associated with this effect.

You can combine blocks by enclosing them into two lines of equality signs, as you can see from the examples below, an effect can also contain further effects.

```
                                {{1-3}}
This is an example for a *single* block effect.

                                {{2-4}}
********************************************************************************

This is an example for a ... wait a second {3 rubberBand}{**multi**} ...
block effect.

* alpha
* beta
* gamma

********************************************************************************
```

This is an example for a *single* block effect.

This is an example for a … wait a second **multi** … block effect.

- alpha
- beta
- gamma

You can put spaces before the definition of a block effect, to prevent github and others from adding the definition to a Markdown paragraph, and thus, to render the script properly.

Narrator

1. we are using the text-to-speech engine of https://responsivevoice.org
2. the narrator voice must be defined within the initial comment of a script
3. use `--{{id}}--` to indicate what is spoken and when

```
                                --{{1}}--
The entire ***Markdown*** paragraph right below the effect definition in double
minus notation is sent to responsivevoice to speak the text out loud. If you
click on the ear button at the navigation panel, then this paragraph gets
rendered at the place where it is defined.

                                --{{3 Deutsch Female}}--
```

```
Markdown ist eine vereinfachte Auszeichnungssprache, die von John Gruber und
Aaron Swartz entworfen und im Dezember 2004 mit Version 1.0.1 spezifiziert ...

                                        --{{4 @Tanja}}--
Первоначально создан в 2004 году Джоном Грубером (англ. John Gruber) и Аароном
Шварцем. Многие идеи языка были позаимствованы из существующих соглашений ...
```

The entire ***Markdown*** paragraph right below the effect definition in double minus notation is sent to responsivevoice to speak the text out loud. If you click on the ear button at the navigation panel, then this paragraph gets rendered at the place where it is defined.

The narrator voice is defined within the initial comment, but you can reset this for the entire section, by adding such a definition directly after the title tag, or you can add a voice directly after the comment number to let your script talk in different languages. Proceed and listen 😉

Markdown ist eine vereinfachte Auszeichnungssprache, die von John Gruber und Aaron Swartz entworfen und im Dezember 2004 mit Version 1.0.1 spezifiziert wurde. Ein Ziel von Markdown ist, dass schon die Ausgangsform ohne weitere Konvertierung leicht lesbar ist.

Первоначально создан в 2004 году Джоном Грубером (англ. John Gruber) и Аароном Шварцем. Многие идеи языка были позаимствованы из существующих соглашений по разметке текста в электронных письмах. Реализации языка Markdown преобразуют текст в формате Markdown в валидный, правильно построенный XHTML и заменяют левые угловые скобки («<») и амперсанды («&») на соответствующие коды сущностей.

Charts

In many cases, a diagram is only used to present some kind of signal paths, some primitive functions, some clusters or point clouds.

You can still generate images, but why not applying some basic kind of ASCII-art to solve the most common tasks.

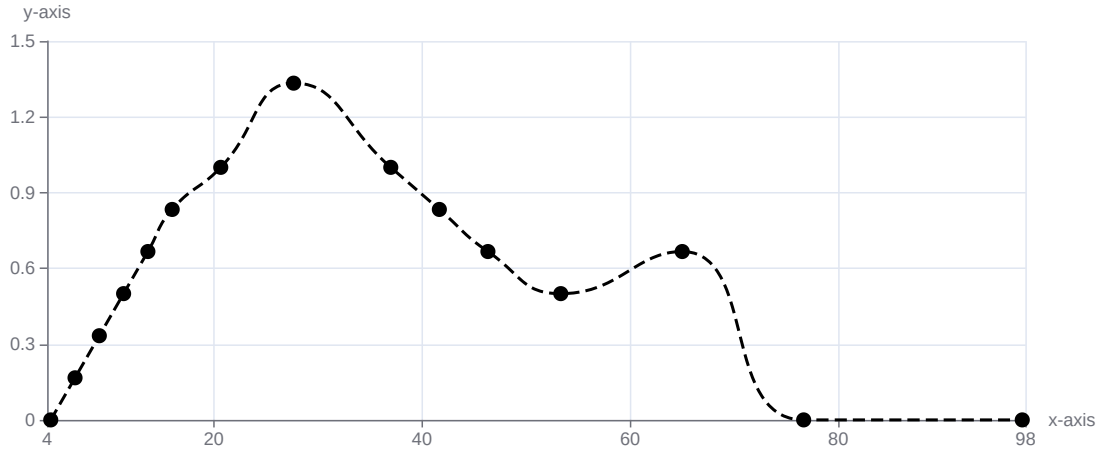
Line-Plots 1

Markdown-format:

```
                                        diagram titel
  1.5 |           *
      |
    y |         *       *
    - |       *           *
    a |     *               *       *
    x |    *                    *
    i |   *
    s |  *
      | *                               *        *
    0 +-------------------------------------------
       2.0              x-axis                100
```

Result:

diagram titel

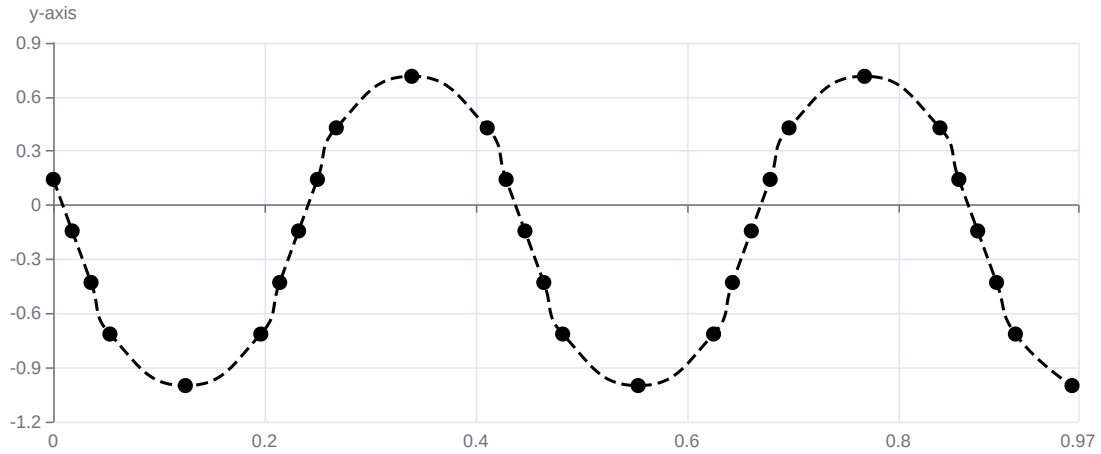


Line-Plots 2

All diagram titles, labels, limits are optional, and if you do not define limits, then the min max values 0 and 1 are used by default.

Markdown-format:

```
    1 |                   *                       *
      |               *       *               *       *
      |*             *       *               *       *
 y-axis | *           *         *           *         *
      |  *          *           *         *           *
      |   *        *             *       *             *
      |     *                      *                     *
   -1 +-----------------------------------------------------
```



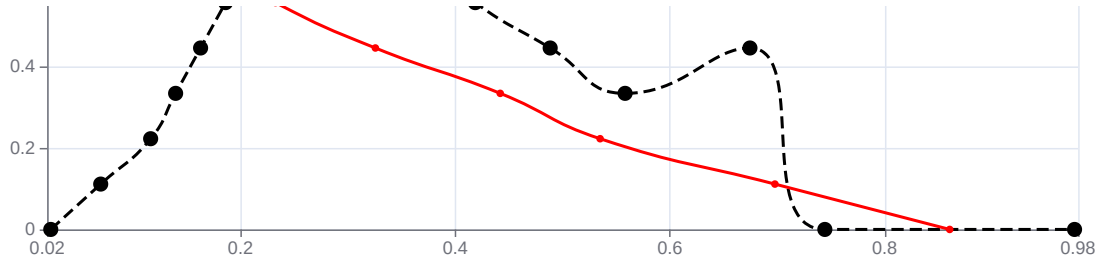
Multi-Line-Plots

Next to stars, you can also use any kind of character to define another line, where the character defines the color. For example an r marks the color red and a b the color blue.

Markdown-format:

```
 | r          *
 |    r
 |       r *      *
 |         * r      *
 |        *      r      *      *
 |       *             r    *
 |      *                  r
 |   *                          r
 | *                              *    r    *
 +----------------------------------------
```



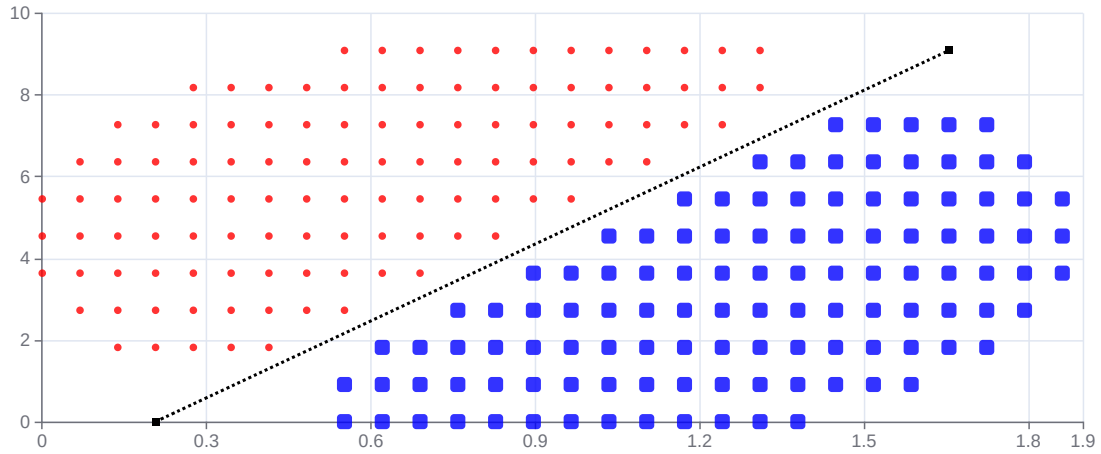


Dot-Plots

If the there are more point with the same character for one x-value, then only dots are plotted. And by using upper and lower case characters you can also define the size of the dots.

Markdown-format:

```
  10 |         rrrrrrrrrrrr    x
     |    rrrrrrrrrrrrrrrr
     |  rrrrrrrrrrrrrrrr  BBBBB
     | rrrrrrrrrrrrrrrr  BBBBBBBB
     |rrrrrrrrrrrrrrr  BBBBBBBBBB
     |rrrrrrrrrrrrr  BBBBBBBBBBBB
     |rrrrrrrrrrr  BBBBBBBBBBBBBB
     | rrrrrrrr  BBBBBBBBBBBBBBB
     |  rrrrr  BBBBBBBBBBBBBBBB
     |         BBBBBBBBBBBBBBBB
     |   x     BBBBBBBBBBBBB
     +-----------------------------
      0                           2
```



Surveys

A script should not define a one-way road to the student! So surveys are required.

Text-Inputs

Similar to text-quizzes, use the following syntax to define a text-survey, where the number of underlines defines the presented line numbers:

```
[[___ ___ ___ ___]]
```

What is your opinion on … :

Enter some text...

Single Choice Vector

And also this kind of survey is similar to a single choice quiz, but in this case numbers within parenthesis are used to define some kind of variable identifier. That is why they do not have to be in order.

```
[(1)] option 1
[(2)] option 2
[(3)] option 3
[(0)] option 0
```

You can only select one option:

- ◯ option 1
- ◯ option 2
- ◯ option 3
- ◯ option 0

Multi Choice Vector

Similar to multi-choice quizzes, you can define multi-choice survey vectors with a number in double square brackets. But, and this is also possible for all other kinds of surveys you can define some kind of variable name with a starting colon.

```
[[:red]]         is it red
[[:green]]       green
[[:blue]]        or blue
[[:dark purple]] no one likes purple ( last chance ;-) )
```

Select some of your favored colors!

- ☐ is it red
- ☐ green
- ☐ or blue
- ☐ no one likes purple (last chance 😉)

Single Choice Matrix

For defining survey blocks you only have to a header row, whose definition is also used by the trailing rows.

Markdown-format:

```
[(:totally)(:agree)(:unsure)(:maybe not)(:disagree)]
[                                                  ] liaScript is great?
[                                                  ] I would use it to make
online **courses**?
[                                                  ] I would use it for
online **surveys**?
```

:totally	:agree	:unsure	:maybe not	:disagree	
◯	◯	◯	◯	◯	liaScript is great?
◯	◯	◯	◯	◯	I would use it to make online **courses**?
◯	◯	◯	◯	◯	I would use it for online **surveys**?

Multi Choice Matrix

I guess, multi-choice blocks are self-explanatory...

Markdown-format:

```
[[1][2][3][4][5][6][7]]
[                     ] question 1 ?
[                     ] question 2 ?
[                     ] question 3 ?
```

Result:

1	2	3	4	5	6	7
▢	▢	▢	▢	▢		question 1 ?
▢	▢	▢	▢	▢		question 2 ?
▢	▢	▢	▢	▢		question 3 ?

ASCII-Art #2

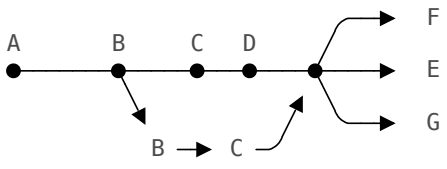
Well, thanks to the great project SvgBob the newest version of LiaScript also has support for some basic ASCII art drawings (not everything is supported yet). Simply use 4 or more backtics to enclose your artwork and draw whatever you want. And as with any other element, you can add some styling within HTML comments at the head of this element.

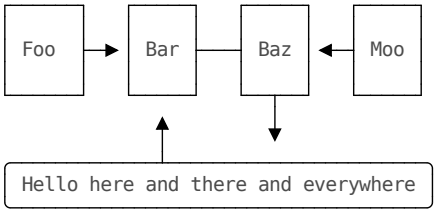
The following examples are taken from the examples on the SvgBob project site.

If you want to use a drawing tool for this, visit the online editor at:

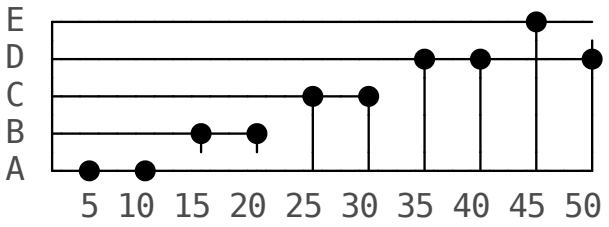
https://ivanceras.github.io/svgbob-editor/

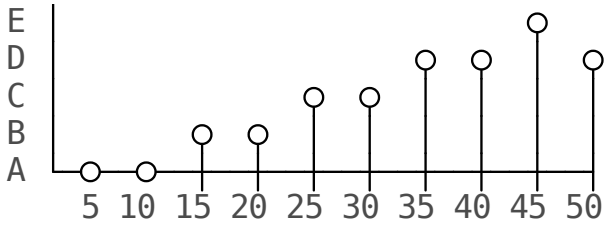
Graphs

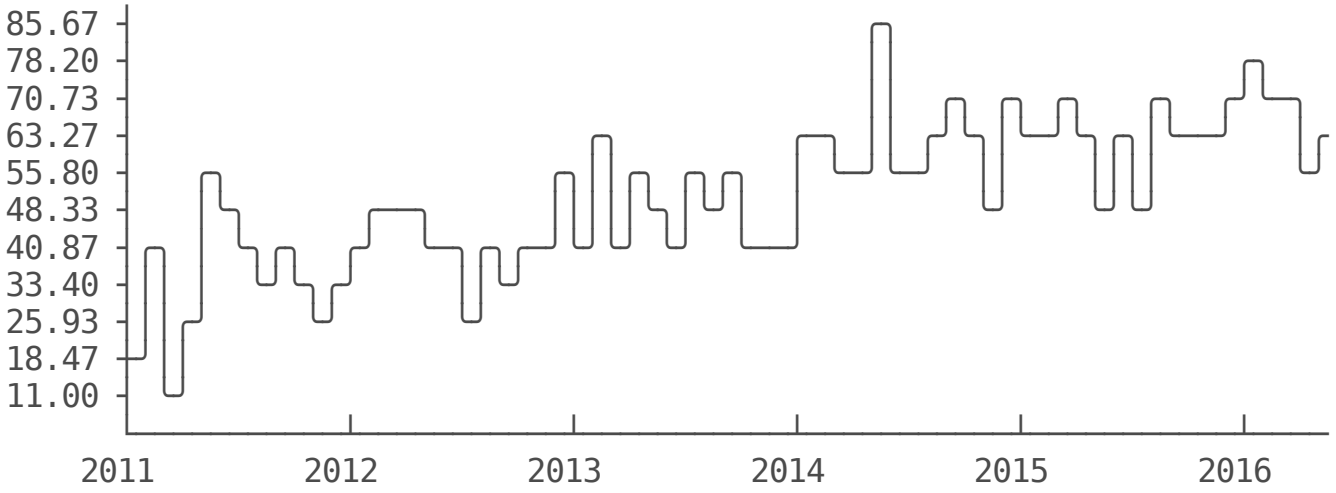




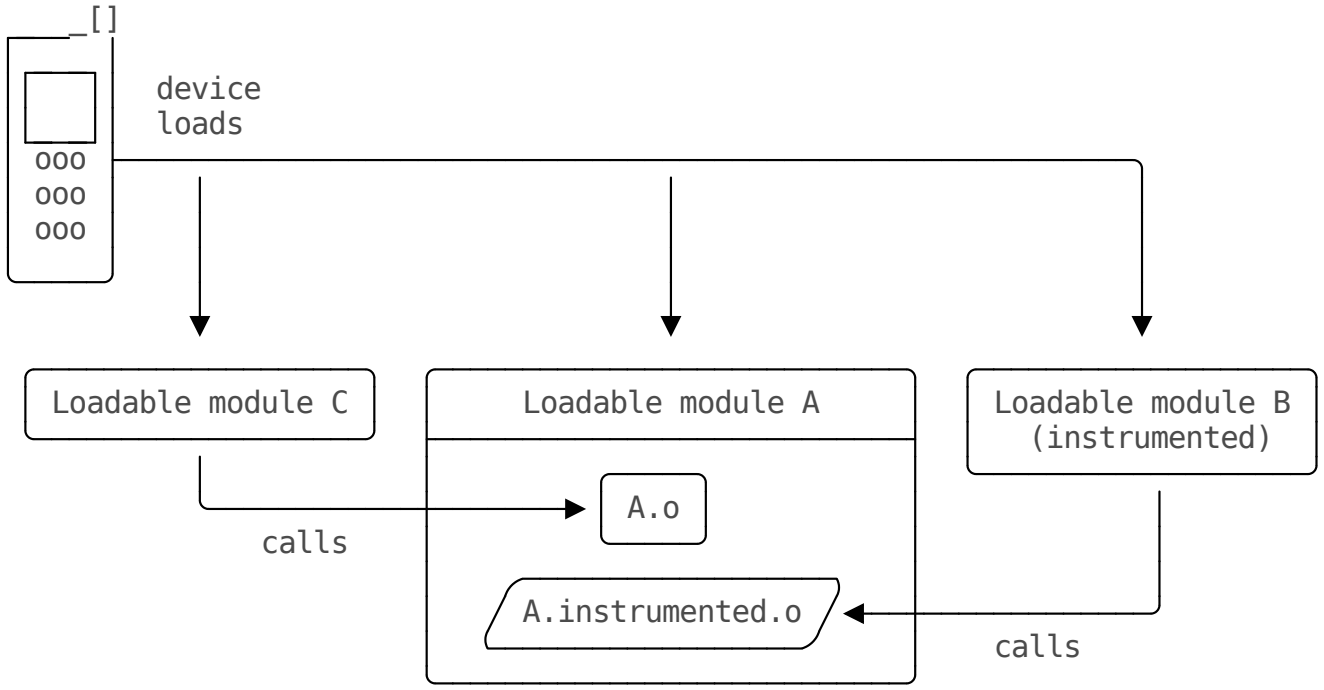
Diagrams

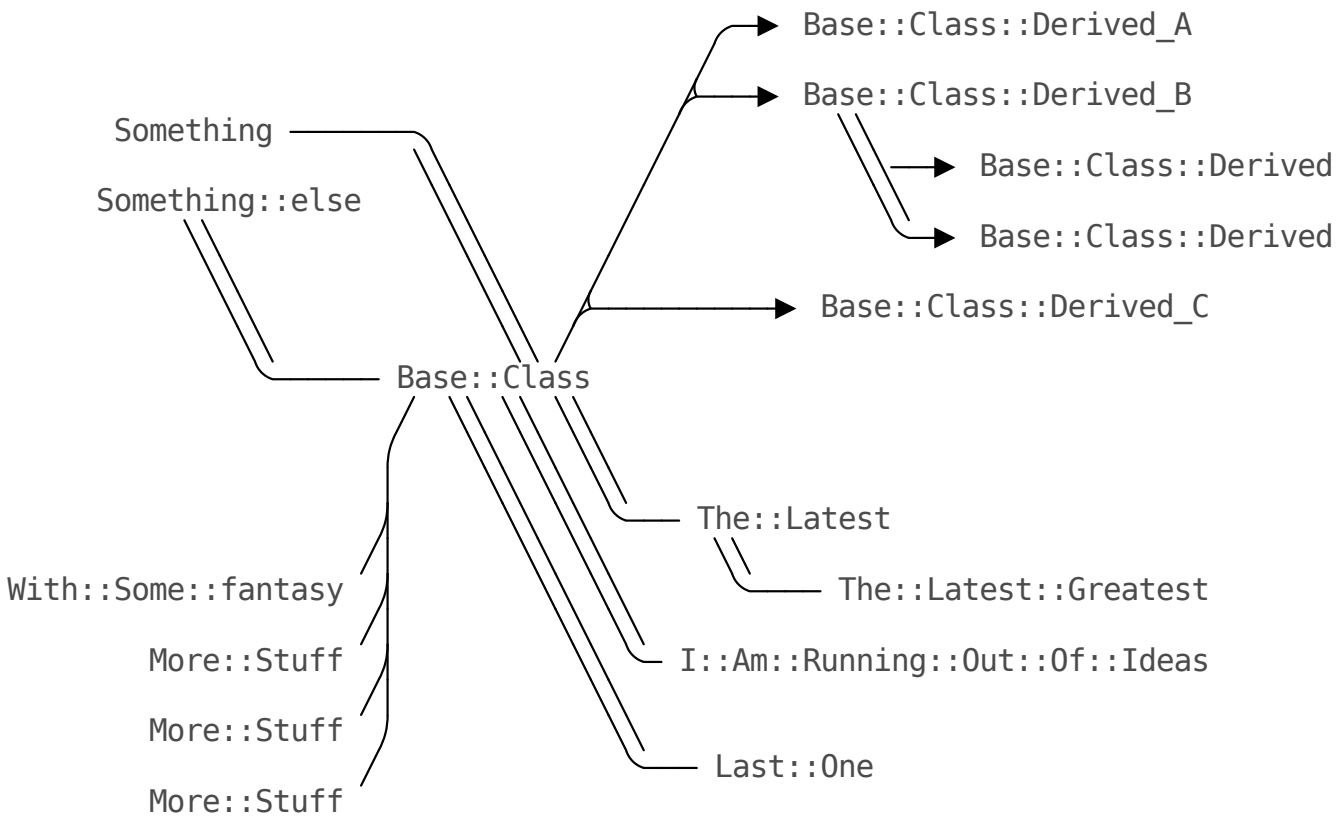




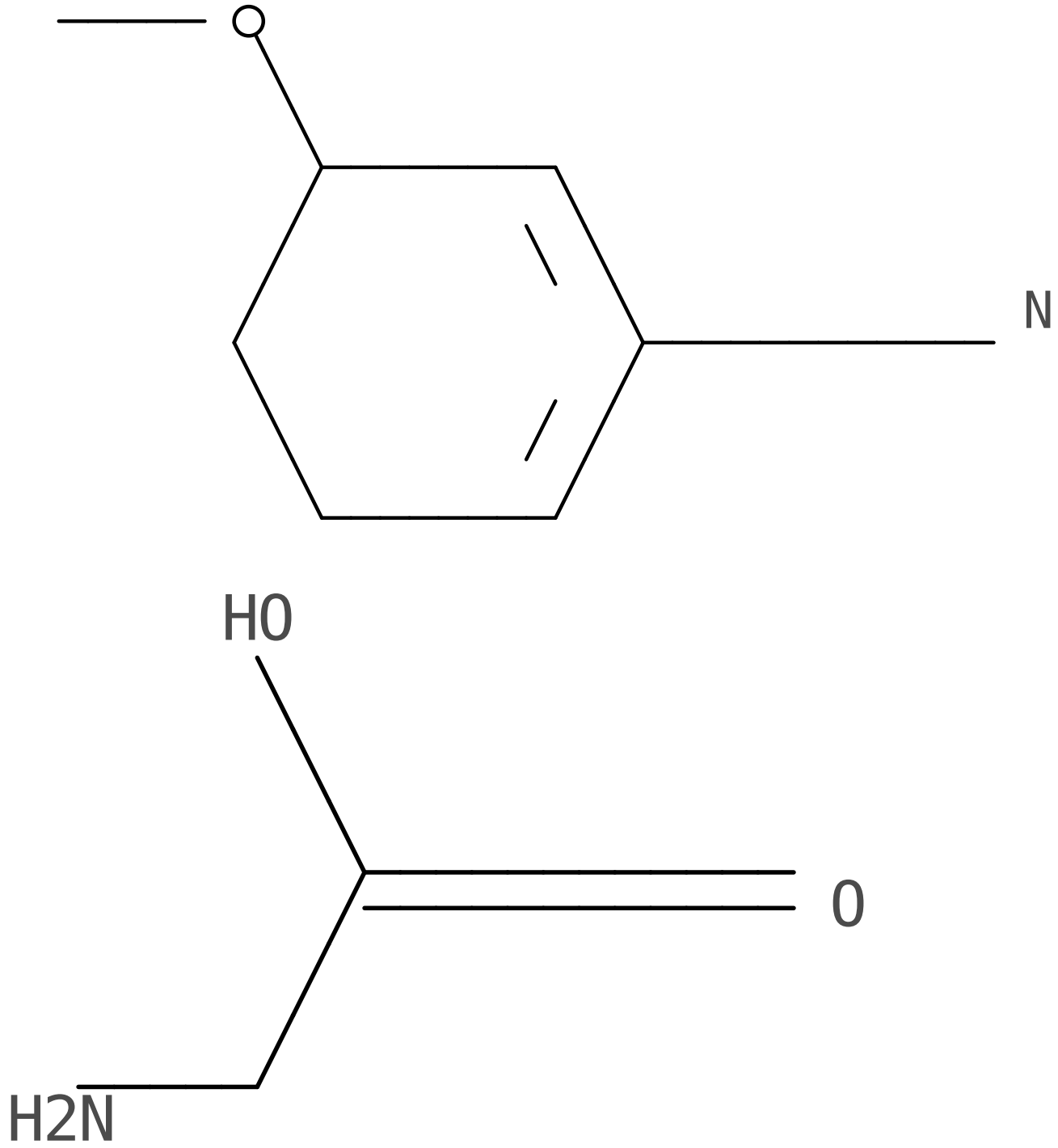


UML

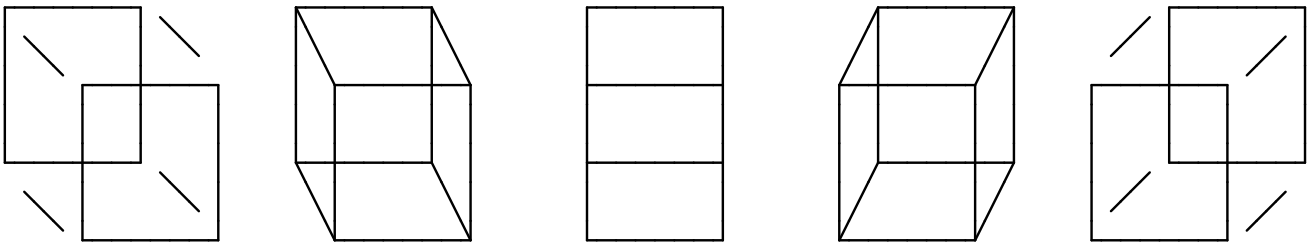


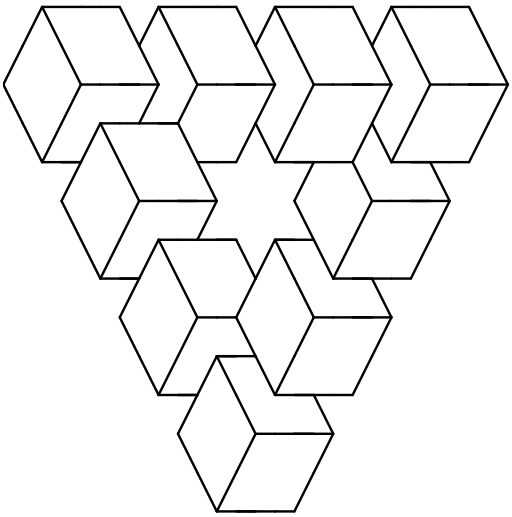


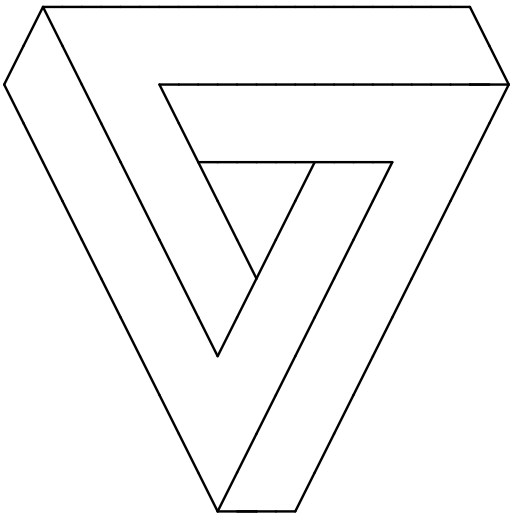
Chemical Structures

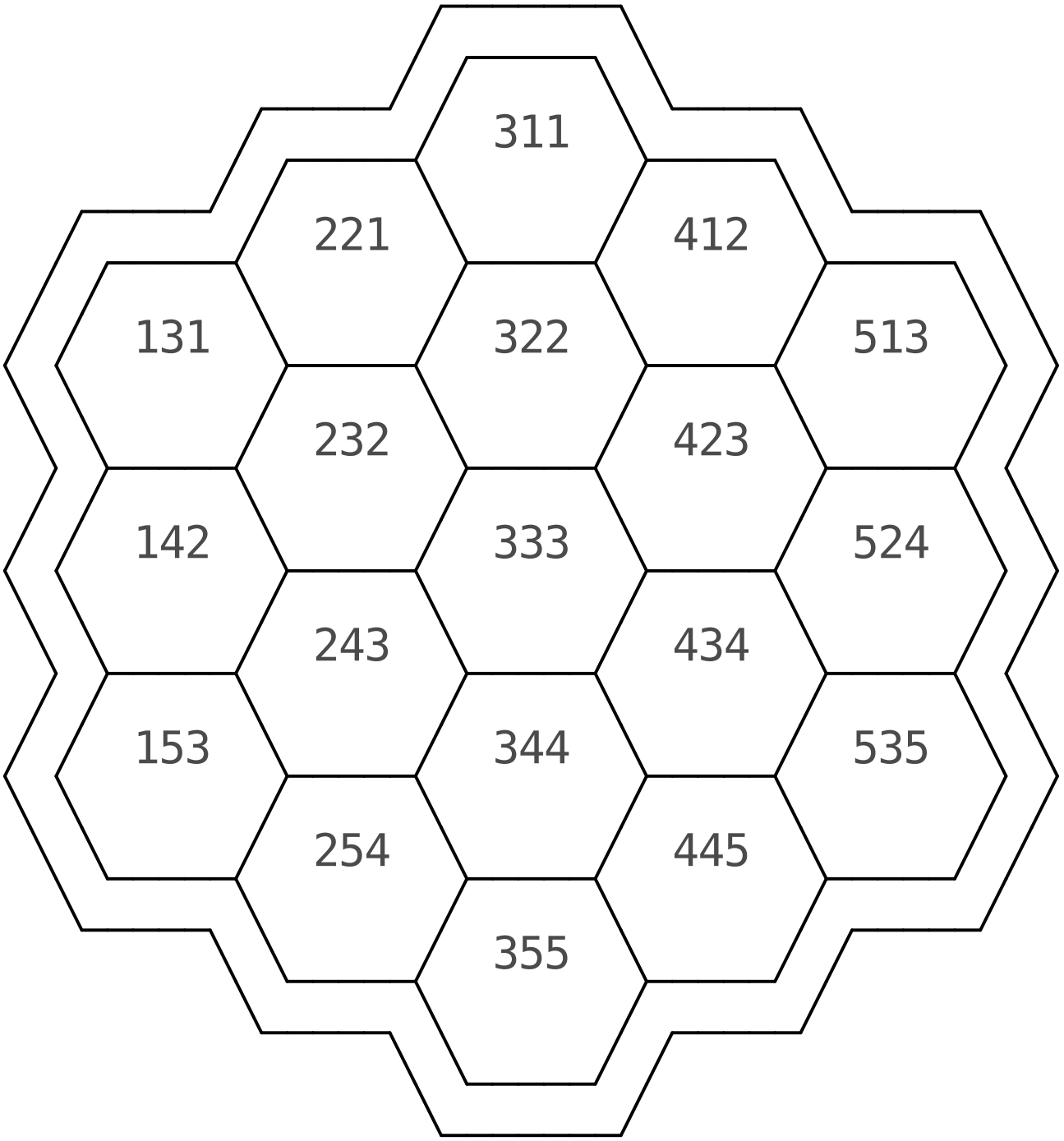


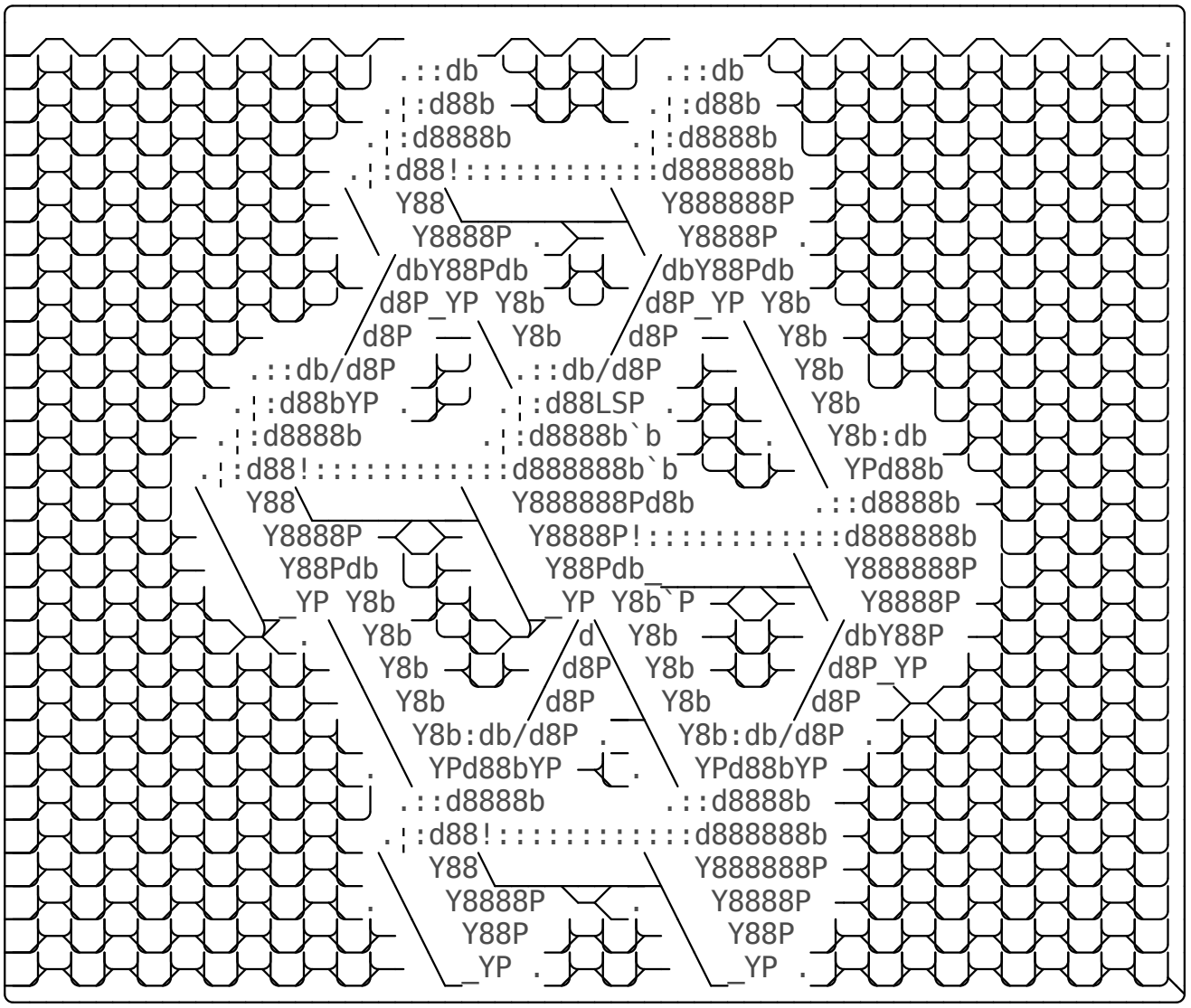
Geometrical Shapes



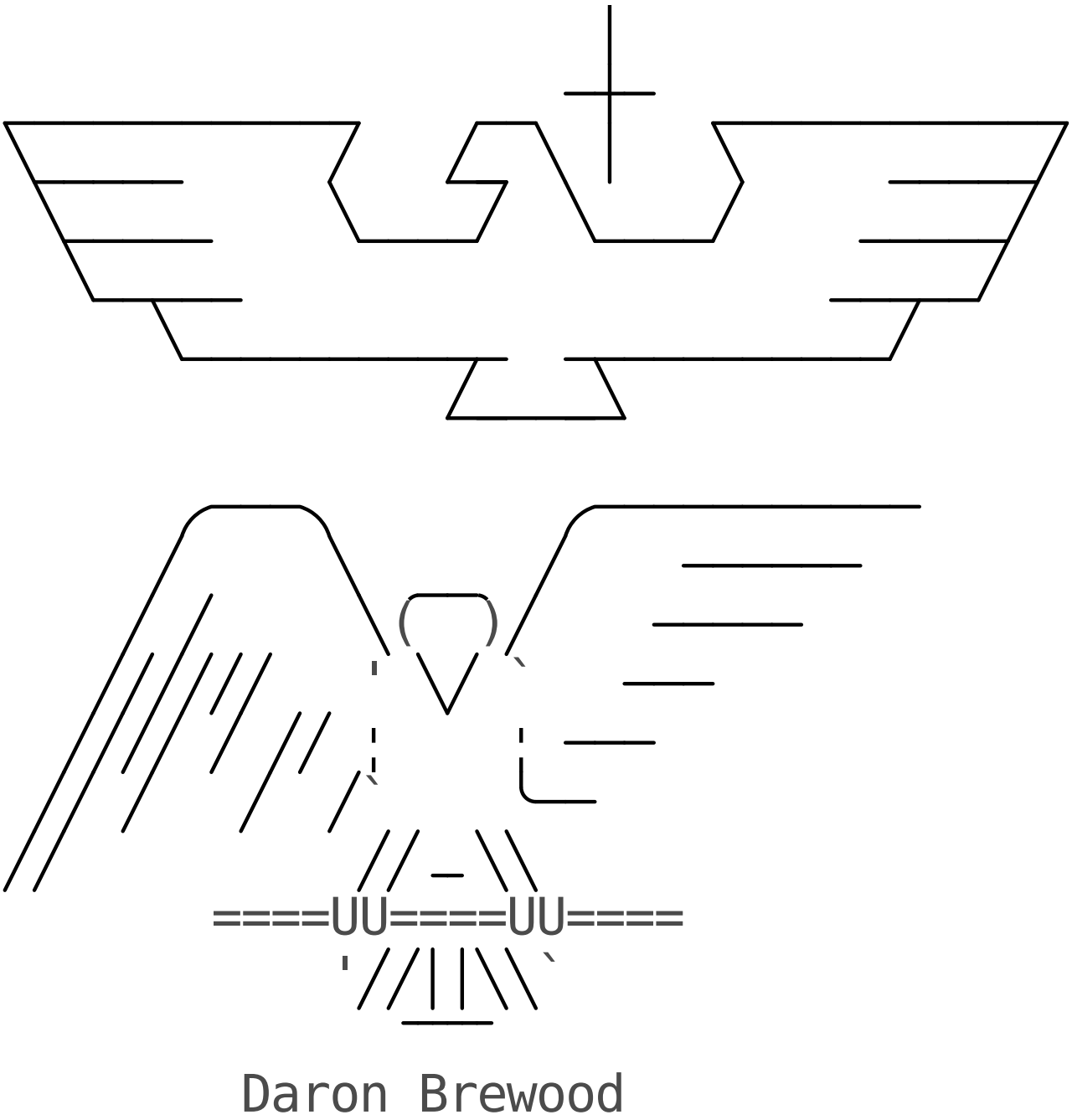





311
221
412
131
322
513
232
423
142
333
524
243
434
153
344
535
254
445
355

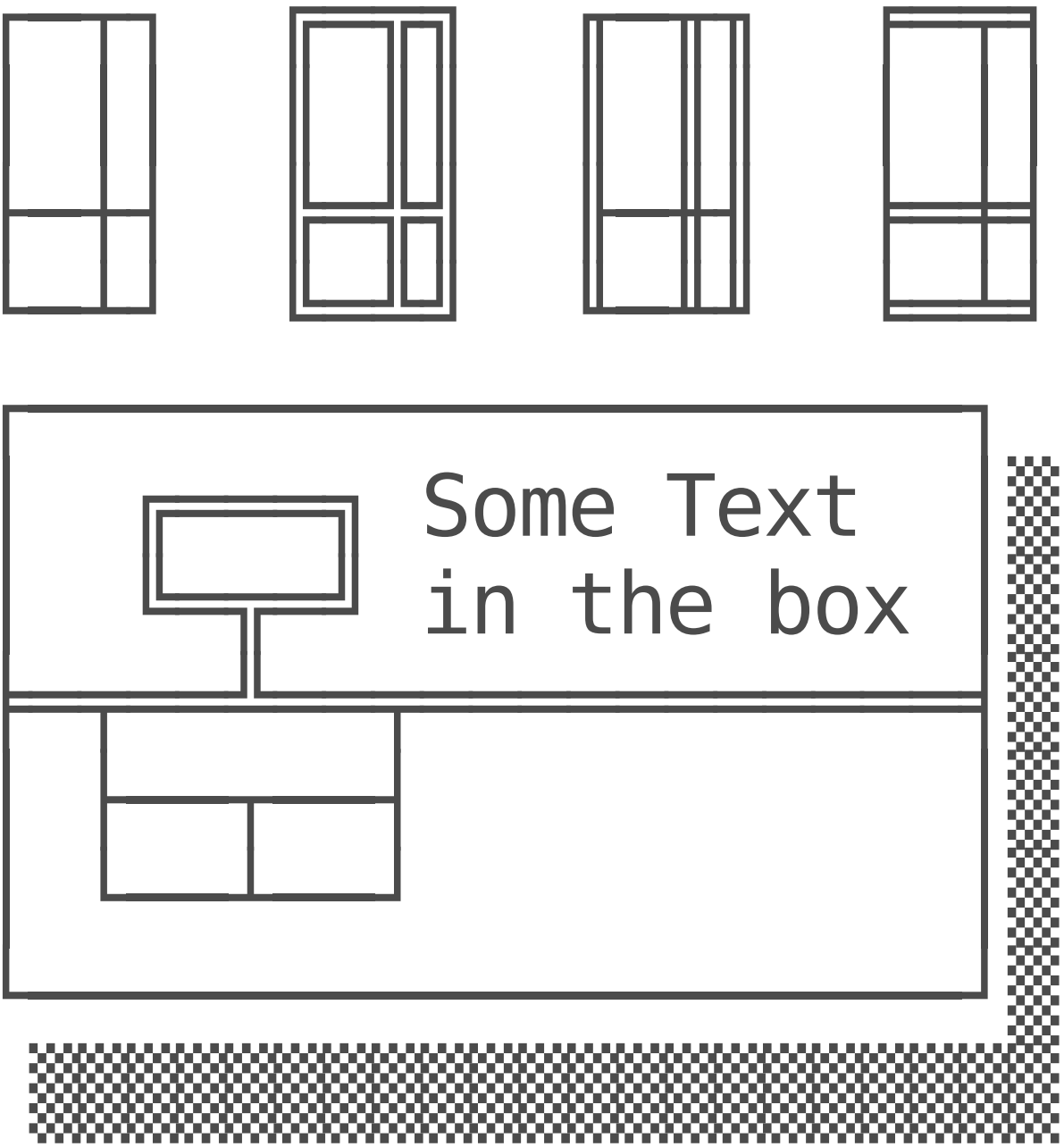


Fun



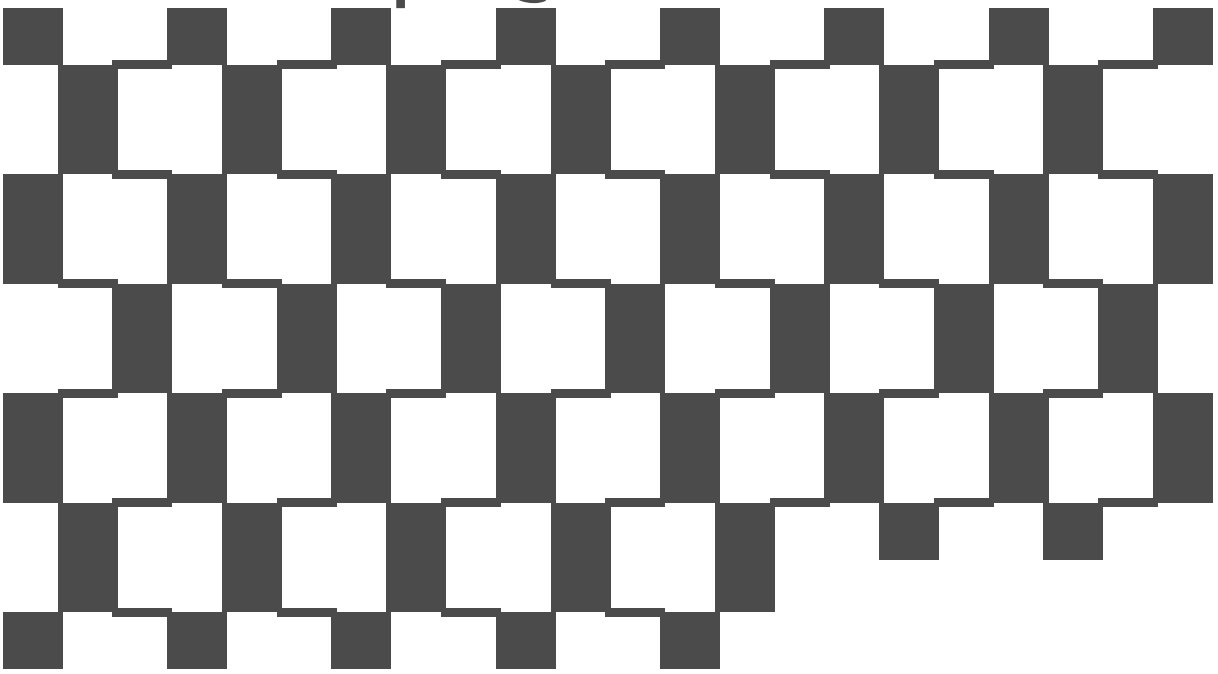
Unicode

And of course, if your ASCII table does not give you enough pleasure, you can also use any kind of Unicode symbol (also within the text).

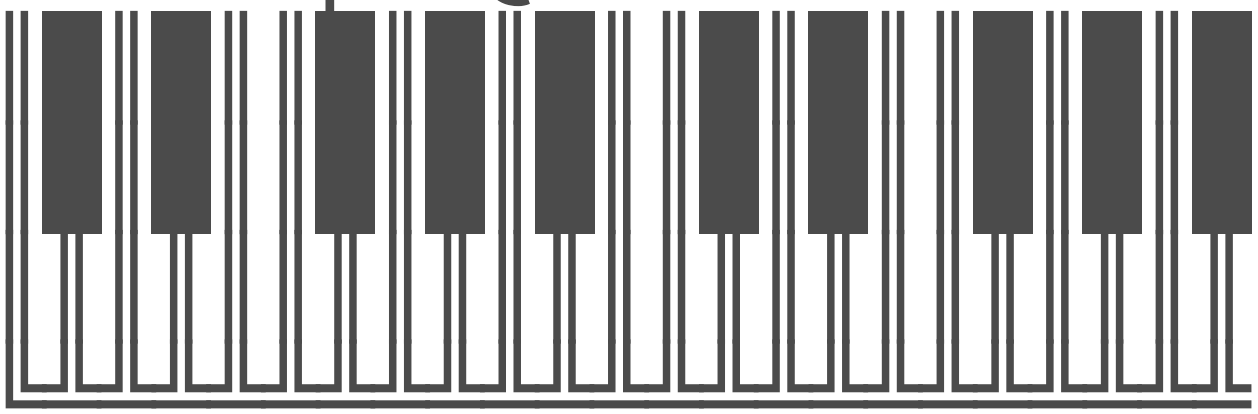


Unicode Art

4 ◇ Sep @tw1tt3rart

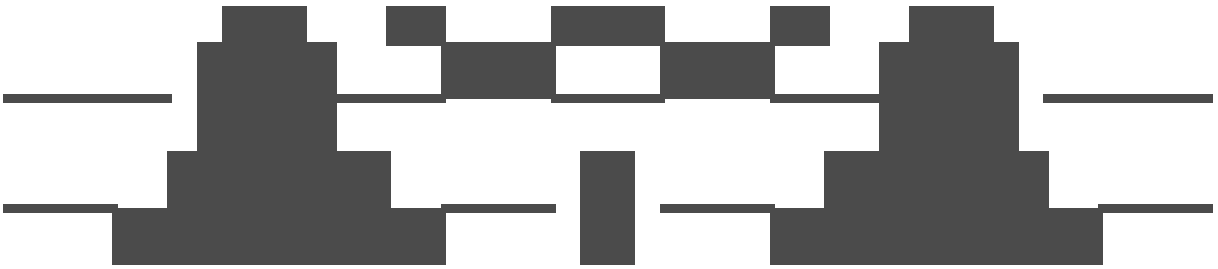


23 ◇ Apr @tw1tt3rart



19 ◇ Jan @tw1tt3rart



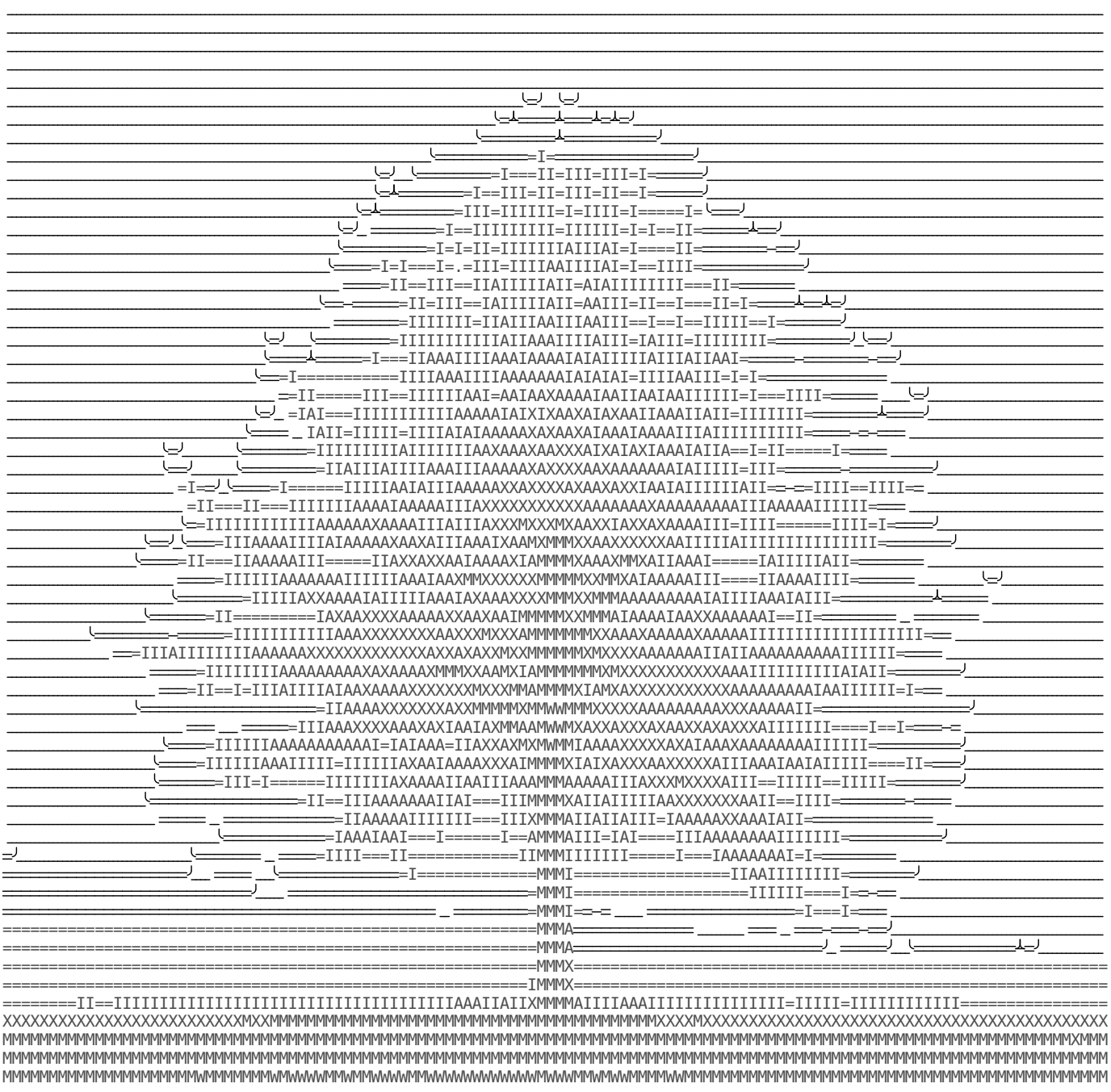


RT
@140artist

Unicode Converter

Image generated with:

http://www.underware.nl/fonts/zeitung_mono/features/ASCII+/#feature_info



Macros

Todo, description of the currently applied macro-syntax.

Future Work

- Better integration with github/gitlab and the versioning of courses
- Automagically analyzed surveys
- Integration of WebGl and a 3D navigation
- Better Inline function-plotter

Contributors and Credit

André Dietrich

Programming paradigm experimenter and creator of liaScript and SelectScript...

Sebastian Zug

The mind in the dark and the man behind the eLab-project …

Karl Fessel

Embedded systems developer, creator or arduinoview, and Markdown evangelist …